

BIA PLATFORM

Multi-Agent AI Operations & Governance Manual

ARCHITECTURE & CAPABILITIES SPECIFICATION

Designed and Built for:

Enterprise Systems Governance and Verification Testing

Technology Stack Summary:

LangGraph, FastAPI, PostgreSQL, Redis, Qdrant DB, OpenTelemetry, Prometheus

Date of Compilation:

July 2026

1. Executive System Introduction

The BIA Platform is a production-grade, stateful, and observable multi-agent AI system designed for corporate database interactions, compliance monitoring, and policy inquiries. By orchestrating specialized LLM agents through a cyclic state graph, the platform automates natural language processing, relational SQL queries, unstructured documentation search (RAG), and data sanitization.

2. Core Platform Capabilities & Operations

- Structured SQL Queries

Allows users to extract information from relational tables using plain English prompts. The system inspects schema constraints, maps references, joins tables, and runs a read-only SELECT statement.

- Unstructured Policy Search (RAG)

Searches corporate handbook markdown files indexed in a vector store. The system reads documents dynamically, embeds queries, fetches matching segments, reranks them locally, and synthesizes answers.

- Secure Mathematical Calculations

Resolves expressions securely via an AST parser tool without running dangerous generic scripts, allowing verified numeric calculation.

- PII & Safety Compliance Scans

Scans response buffers before returning them. It intercepts, redacts, and masks sensitive data, such as emails, credit cards, and password hashes, returning sanitized blocks.

3. The 8 Specialized Agents

* Intent Router Agent

Classifies user queries to decide execution paths (e.g. RAG, SQL, API, General, Unsafe) and intercepts SQL injections.

* Task Planner Agent

Constructs a checklist of required operations, revising the plan in case of execution anomalies.

* SQL Database Agent

Translates questions into read-only SQL SELECT queries and runs them against PostgreSQL.

* Retrieval Agent

Queries the Qdrant vector database and reranks the results via a Flashrank cross-encoder.

* API Client Agent

Triggers outbound integration web requests via MCP tools to fetch live details.

* Reflection Agent (QA)

Audits tool execution results. If errors occur, it loops back to the Planner with critiques.

* Safety Compliance Agent

Scans outputs, redacting credit cards, password hashes, and emails to preserve data privacy.

* Report Compilation Agent

Gathers all intermediate steps and compiles a final markdown report for the user.

4. Model Context Protocol (MCP) Integration

The Model Context Protocol (MCP) is utilized to decouple agent core logic from tool execution. Rather than configuring tool scripts locally in the agent runtime, all tools (SQL, Retriever, Filesystem, Calculator, REST API, Memory) are exposed as standard endpoints by a dedicated MCP server.

Endpoints and Call Lifecycle:

1. GET /mcp/tools: Returns JSON tool schema definitions so agents understand required parameters.
2. POST /mcp: Accepts argument schemas from agents, validates permissions, runs tools, and returns standard outputs.

Local Fallback Design:

To ensure high availability, if the gateway loses connection to the MCP microservice server, agents automatically default to local direct imports of the tool libraries to avoid downtime.

5. RAG Technical Architecture

The Retrieval-Augmented Generation (RAG) system is built on a modern decoupled architecture:

- **Vector Store:** Qdrant Database hosts vector collections containing segmented corporate manuals.
- **Indexing Pipeline:** Python loaders segment markdown documents into semantic nodes, generate OpenAI embeddings (text-embedding-3-small), and upload points.
- **Query Processing:** Combines cosine-similarity vector matches with an advanced local Flashrank reranker. Reranked contexts are fed to the LLM for precise citations, preventing hallucinations.

6. Observability & Monitoring Controls

Production runtime requires active health and latency tracking across the microservices:

- **Prometheus Integration:** Exposes real-time endpoints collecting token counts, execution costs, retry loop intervals, and gateway HTTP counts.
- **Grafana Visualizations:** Displays latency logs, total tokens consumed, cost tracking, and execution metrics on a clean dashboard.
- **LangSmith Tracing:** Collects deep token traces and agent transition graphs, facilitating prompt and edge testing.